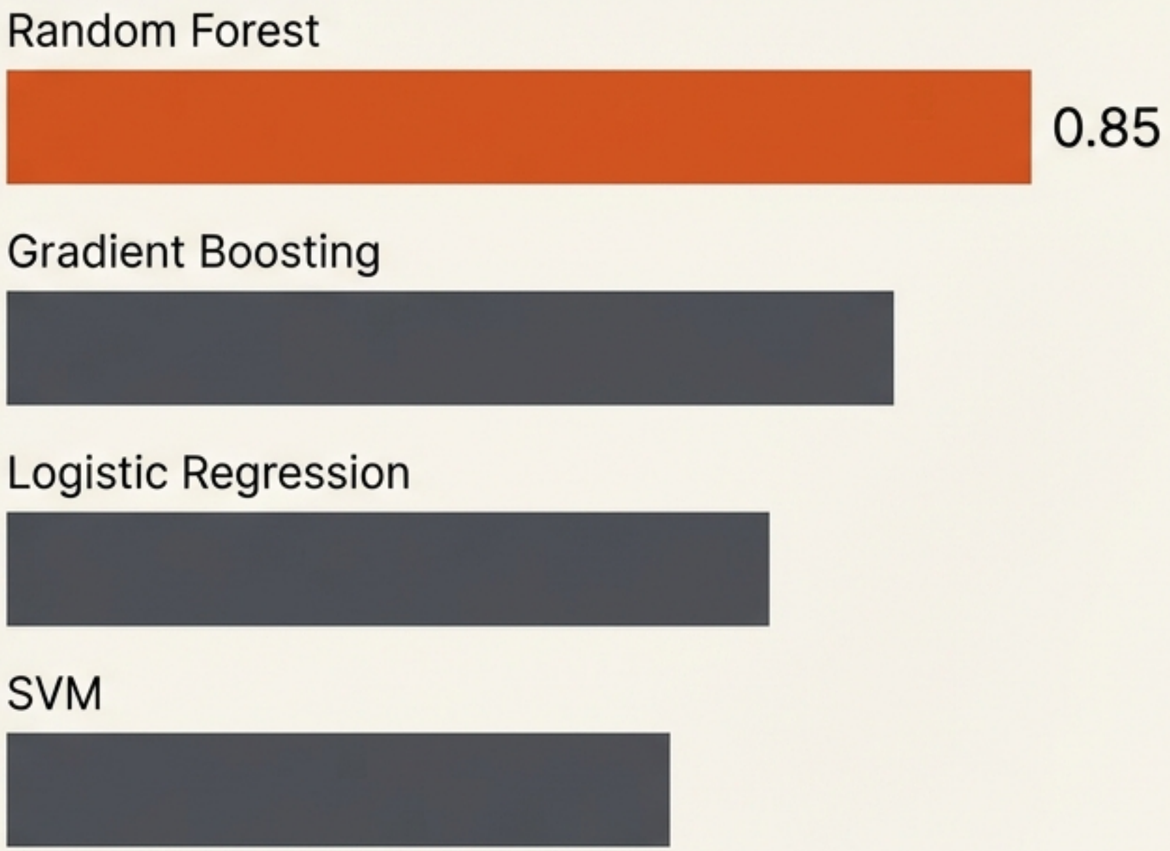
Mastering Random Forest: A Practical Guide to Hyperparameter Tuning

From out-of-the-box baselines to optimised models using Grid and Random Search

Out-of-the-Box Power

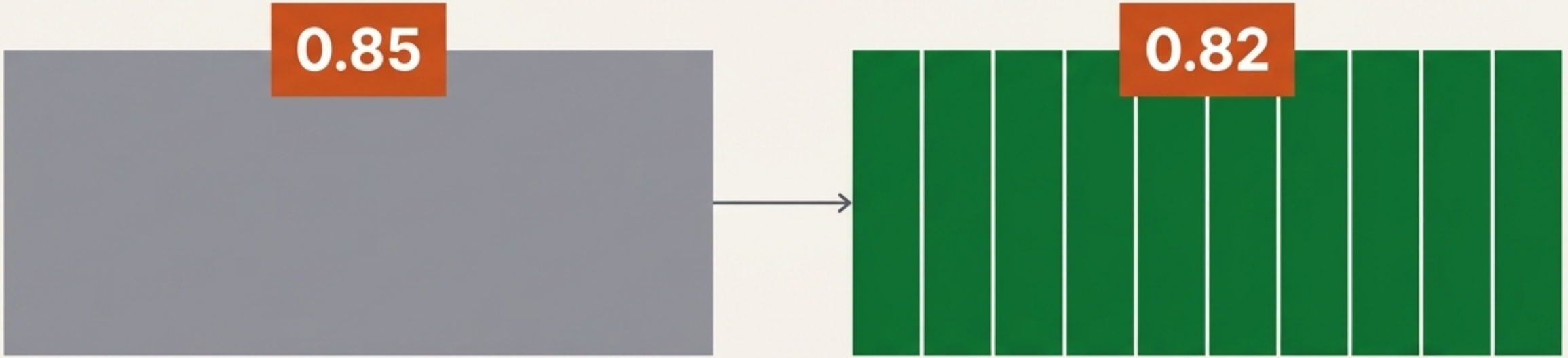
Random Forest consistently ranks among the top-performing algorithms directly out of the box, without any hyperparameter tuning.

Dataset: Heart Disease Classification
Total Patients: 303 (242 Train / 61 Test)
Single Split Accuracy: ~0.85



The Reality Check: Cross-Validation

A single train-test split can create an illusion of high performance. Applying a 10-fold cross-validation provides a truer, slightly more conservative representation of real-world accuracy.



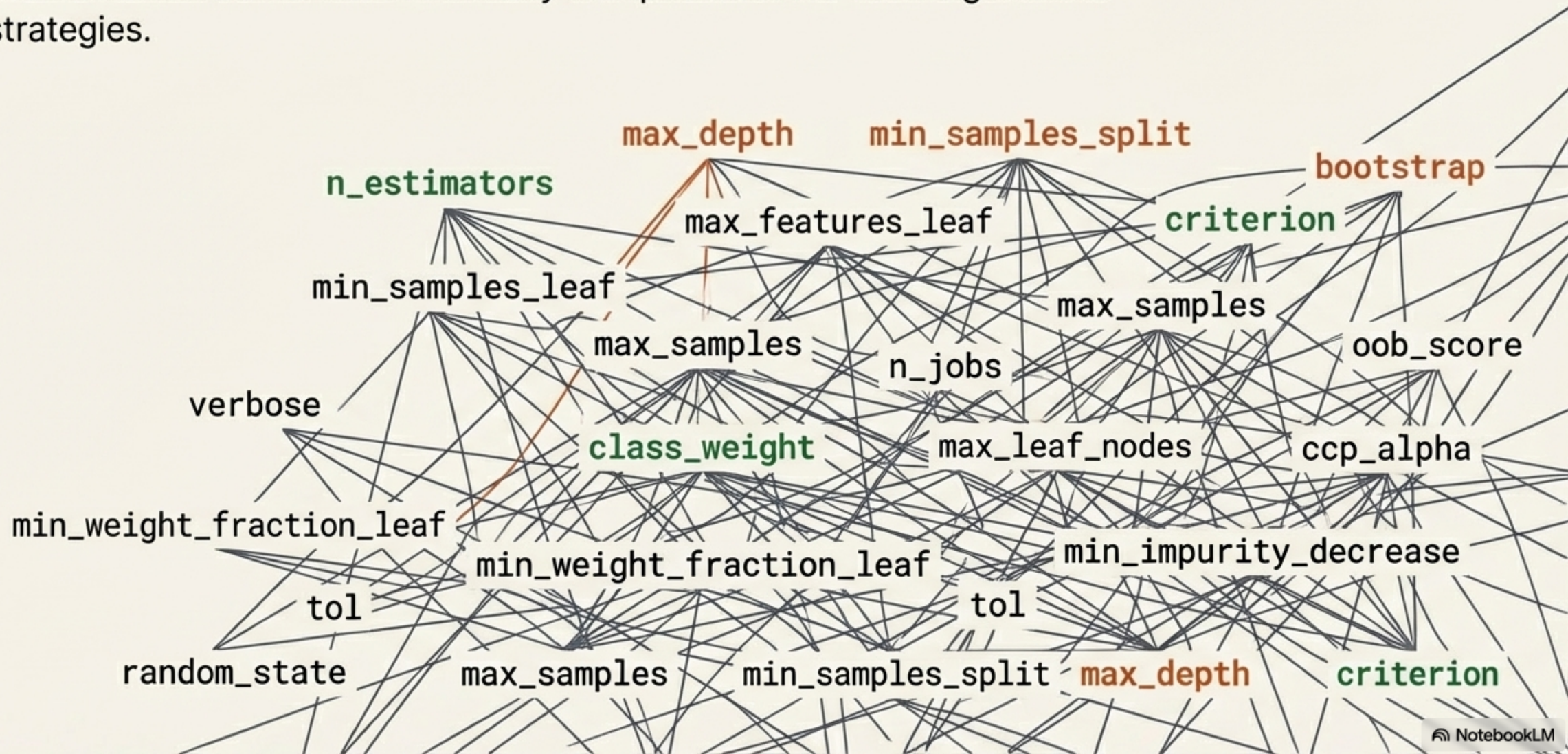
True Baseline Accuracy: 0.82

The Power of a Single Tweak

Hyperparameter tuning bridges the gap between a good model and a great one. Adjusting just one parameter significantly improves our cross-validated baseline.

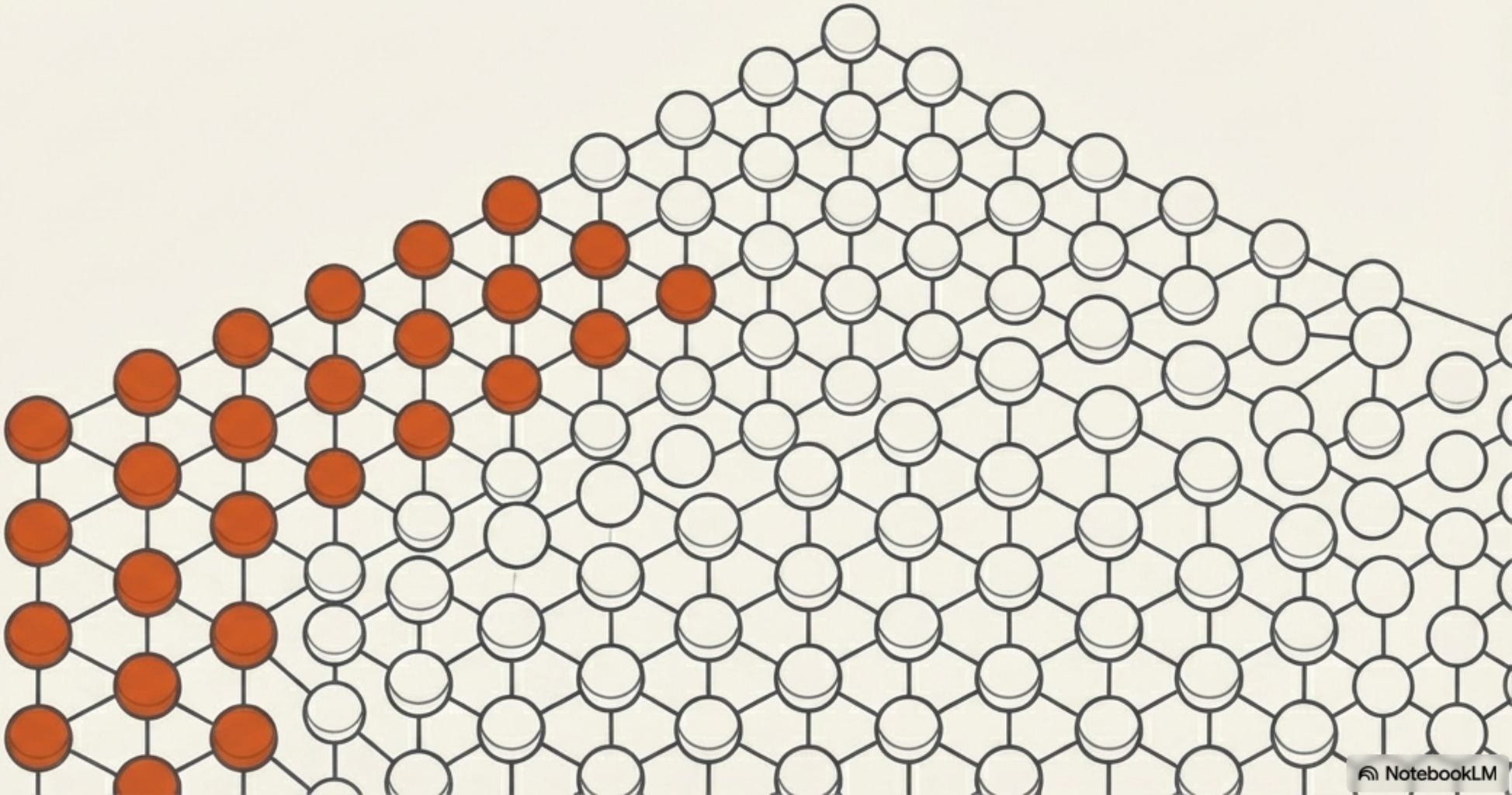
The Optimisation Challenge

Random Forest has approximately 25 distinct hyperparameters. Finding the perfect mathematical combination manually is impossible. We need algorithmic search strategies.



Method 1: GridSearchCV

The exhaustive approach. GridSearchCV acts as an N-dimensional table. Provide specific values for your parameters, and it methodically tests every single possible combination.



The Computational Weight of the Grid

Exhaustive search guarantees thoroughness, but the computational load scales exponentially.

$$[\,4\text{ values}\,] \times [\,3\text{ values}\,] \times [\,3\text{ values}\,] \times [\,3\text{ values}\,] = 108 \text{ Combinations}$$

$$108 \text{ Combinations} \times 5 \text{ CV Folds} = 540 \text{ Total Fits}$$

Parameter Breakdown

n_estimators:
4 values

---

max_features:
3 values

---

max_depth:
3 values

---

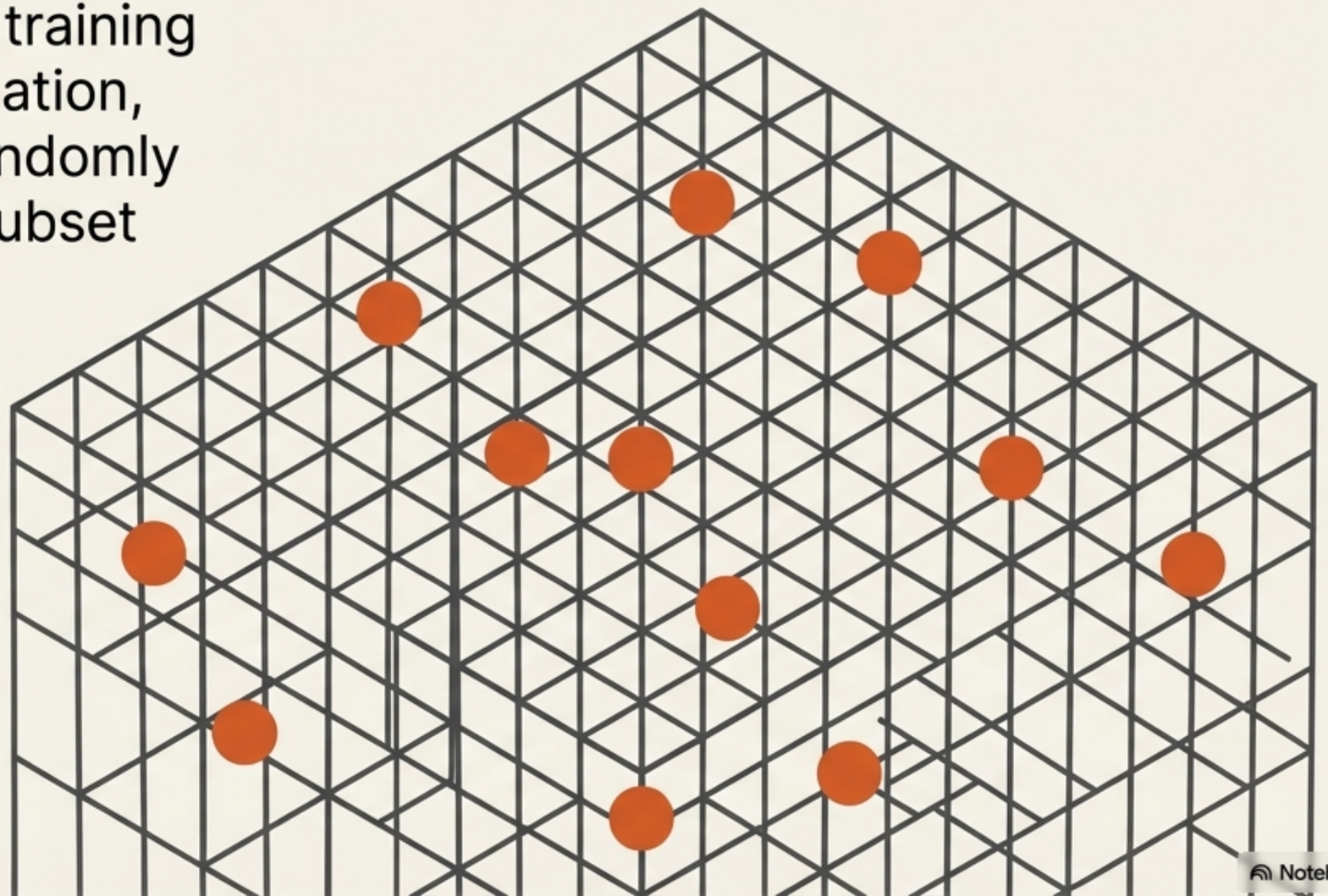
max_samples:
3 values

The Limitations of the Exhaustive Grid

While GridSearchCV successfully identified an optimal parameter set achieving a 0.83 score, it is incredibly **slow**. On massive datasets with deeper parameter grids, the exhaustive nature becomes computationally prohibitive.

Method 2: RandomizedSearchCV

The fast track. Instead of training on every possible combination, **RandomizedSearchCV** randomly samples a user-defined subset of the parameter space.



Speed vs. Absolute Perfection

By restricting the search to just **10 random** candidates, the computational time drops dramatically. This newfound speed allows us to test a much broader range of parameters simultaneously.

10 **Candidates** × **5** **CV Folds** = **50** **Total Fits**

Expanded Scope

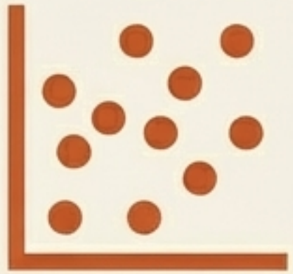
Quickly test added parameters like bootstrap, min_samples_split, and min_samples_leaf without stalling your machine.

The Catch: Because it skips combinations, it may miss the absolute optimum (scoring 0.81 in our demo).

Side-by-Side Comparison

GridSearchCV

- Exhaustive evaluation
- Computationally heavy / Slower
- Guarantees the best combination within the grid

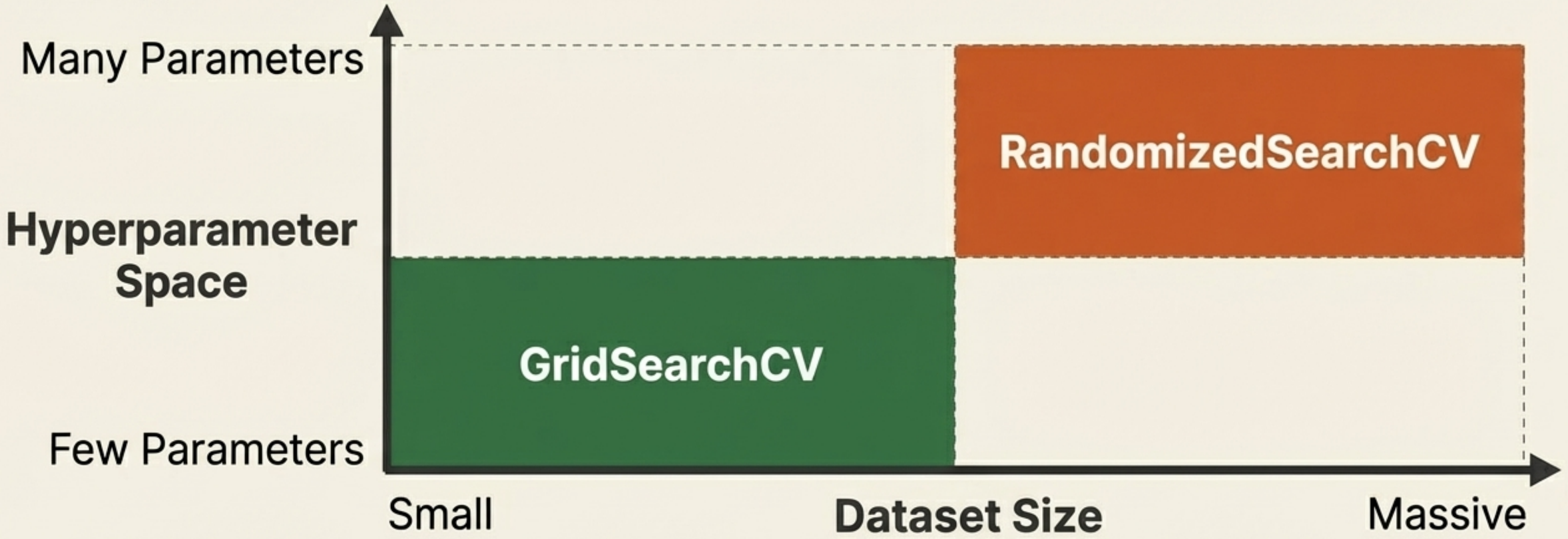


RandomizedSearchCV

- Sampled evaluation
- Exponentially faster
- Delivers near-perfect results in a fraction of the time

The Decision Matrix

Choose your tuning strategy based on your data scale and computational budget.



Key Takeaways

1

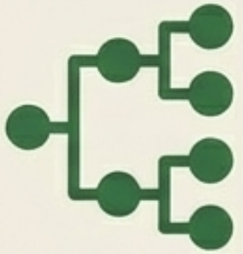
Always Cross-Validate

Single train-test splits create illusions. Use 10-fold CV for a true performance baseline.

2

Tuning Unlocks Peak Performance

Random Forest is strong out-of-the-box, but targeted hyperparameter adjustments separate good models from great ones.



3

Match Method to Scale

Use Grid Search for absolute precision on smaller datasets, and rely on Random Search for speed and scale on massive problems.